Relatório 2 - Prática: Linguagem de Programação Python (I)

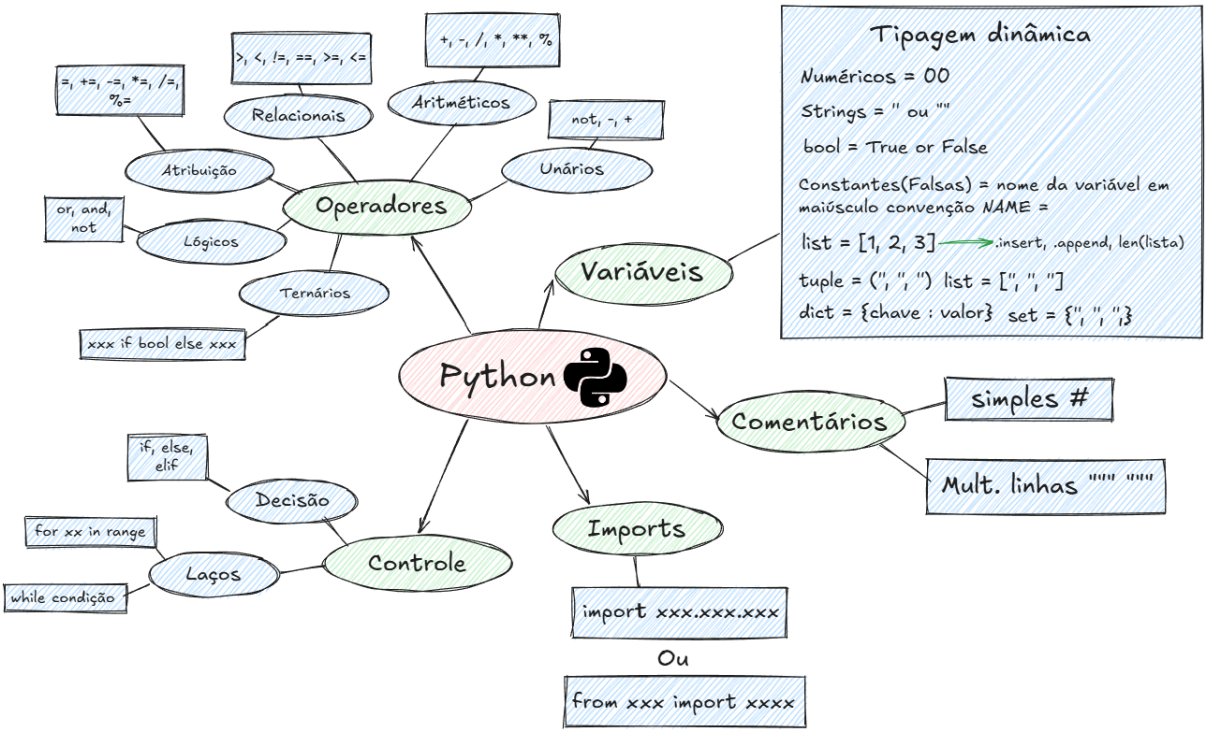
Eduardo Andrei Staudt

1. Introdução

O objetivo desta aula foi ter um primeiro contato e aprender o básico sobre a linguagem de programação *Python*. A tarefa consistia em assistir a dois vídeos e abstrair diversos conceitos básicos e intermediários da linguagem como convenções, operadores, variáveis, estruturas de controle, funções, lambdas, classes, além de um pouco de OO(Orientação a Objetos) e Programação Funcional.

2. Desenvolvimento

Imagem 1 - Mapa mental aula 1.



Fonte: Autoria própria feita no Excalidraw.

Na Imagem 1 acima se tem um mapa mental com os conteúdos abstraídos por mim ao assistir o primeiro vídeo em que foram apresentados os seguintes conceitos da linguagem de programação *Python*:

- **Variáveis e tipos:** na linguagem *Python* utiliza-se tipagem dinâmica que determina o tipo na variável em tempo de execução, sendo baseada no valor

a ela atribuído, existem diversos tipos como os numéricos, *strings*, booleanos e coleções mais especificados abaixo:

- **Numéricos**: os principais tipos são *int*(inteiros), *float*(números flutuantes), *complex*(para números complexos).
- ***Strings***: que são variáveis de texto do tipo str que podem ser declaradas com ‘’ ou “”.
- **Booleanas**: são variáveis de decisão *True*(Verdadeiras) ou *False*(Falsas).
- **Constantes**: o *Python* não possui constantes verdadeiras mas usa-se um padrão para nomeá-las sendo ele colocado em maiúsculo tipo: NAME = 122(falsa constante).
- **Coleções**: são variáveis que permitem guardar múltiplos dados elas são *list, tuple, dict e set* a tabela abaixo mostra a sintaxe e algumas diferenças entre elas:

Tabela 1 - Diferenças entre os tipos de coleções.

Coleção	Sintaxe	Acessáveis por idx	Mutável	Aceita duplicatas
Lista (list)	[1, 2, 3]	Sim	Sim	Sim
Tupla(tuple)	(1, 2, 3)	Sim	Não	Sim
Conjunto(set)	{1, 2, 3}	Não	Sim	Não
Dicionário(dict)	{chave : valor}	Sim (por chave)	Sim	Nos valores

Fonte: Autoria própria feita no Excalidraw.

- **Operadores**: Operadores nos permitem fazer operações com dados, existem muitos tipos e eles são classificados da seguinte forma:
  - **Atribuição**: servem para guardar valores em variáveis (=, +=, -=, *=, /=, %=).
  - **Relacionais**: comparam valores e retornam *True* ou *False* (>, <, >=, <=, ==, !=).
  - **Unários**: atuam sobre um único valor (*not*, +, -).
  - **Aritméticos**: usam múltiplos operandos para fazer cálculos (+, -, *, /, **, %).
  - **Lógicos**: combinam condições (*and, or, not*).
  - **Ternário**: faz uma condição curta em uma linha (x *if* condição *else* y).
- **Estruturas de Controle**: Essas estruturas permitem fazer decisões e repetições dentro do código, no vídeo foram apresentadas as seguintes:
  - **Decisão**: Para tomar decisões do tipo se, senão no *Python* usamos o *if, else e elif o if* é a condicional principal(se) o *elif* é uma condição

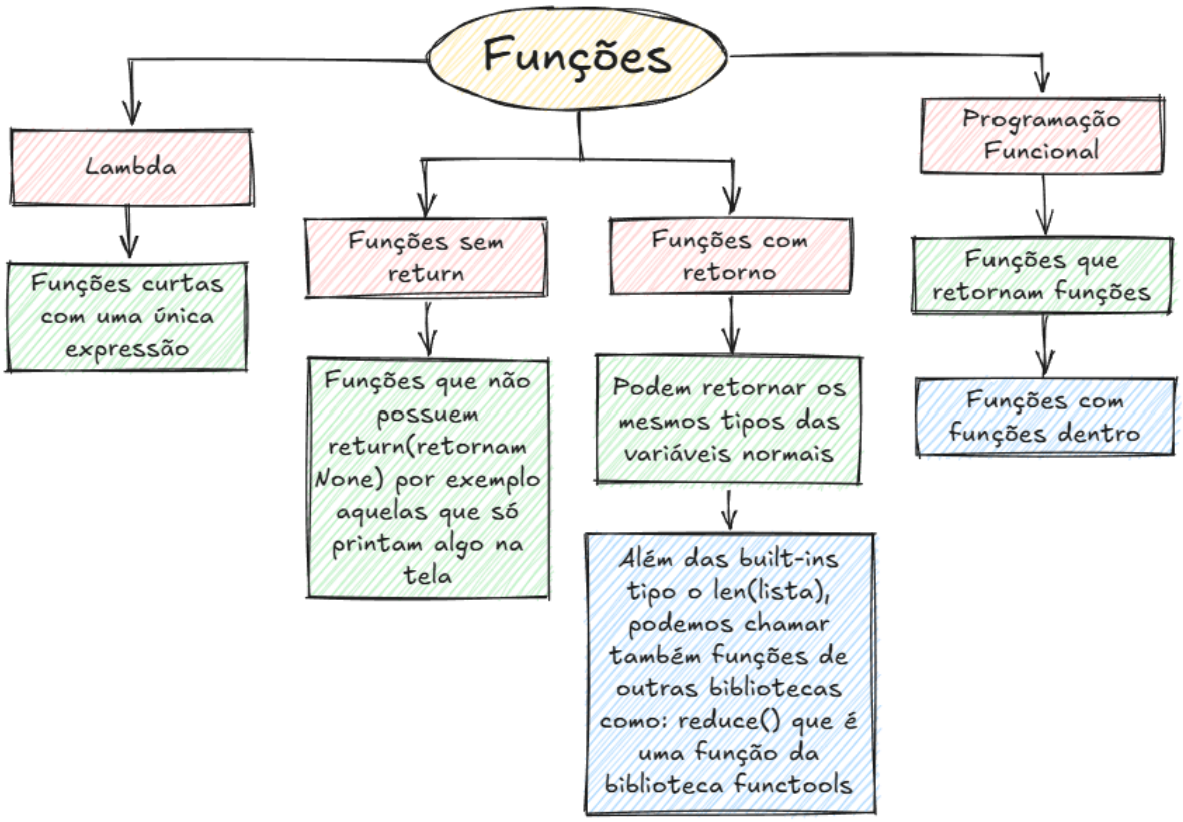
seguinte e o *else* é para caso nenhuma condição anterior tenha sido aceita.

- **Laços**: São utilizados para repetir múltiplas vezes os blocos de código dentro delas, existem dois tipos *for*(*for* xxx *in* xxx:) em que é passado uma variável para percorrer uma outra e o *while*(*while* condição:) que é executado enquanto a condição passada a ele for verdadeira. Neles podem ser o *continue*(pula para a próxima iteração) e o *break*(sai totalmente do laço).

- **Imports**: o *Python* nos permite puxar instruções de outros arquivos, pacotes e bibliotecas o que facilita a reutilização do código.
- **Comentários**: No *Python* dá para fazer comentários no código de duas maneiras **#** texto ou **“””** texto **“””**(*String* descartada) essa segunda forma permite comentários em blocos geralmente ela é mais usada para documentação.

Abaixo vou abordar os conteúdos do segundo vídeo, que já possuem um nível mais complexo do que na aula passada, como funções, retornar funções, funções dentro de funções, lambda e classes, estes são conhecimentos de extrema importância pois geralmente são eles que diferenciam um software de qualidade em vários sentidos como organização, custo, tempo de processamento.

Imagem 2 - Mapa mental sobre funções.



Fonte: Autoria própria feita no Excalidraw.

Na imagem 2 fiz um mapa mental com os principais conceitos de funções que absorvi durante o vídeo, para se declarar uma função utiliza-se a palavra reservada ***def***:

- **Funções sem retorno**: Estas são funções que servem mais para mostrar algo no terminal, definir valores (*setters*) e mutação de estado, estas funções sem retorno retornam *None*(ou void em outras linguagens) do *Python*.
- **Funções com retorno**: Este tipo de funções é muito utilizado na programação, elas utilizam o return que pode ter diversos tipos o que é muito útil por permitir trabalhar com a saída da função, o *Python* possui funções *built-ins* que já estão nele e podem ser utilizadas sem a necessidade de imports, além disso existem diversas outras funções que podem ser importadas de outros módulos e bibliotecas como a *reduce*() do *functools* utilizada no vídeo.
- **Lambda**: Existem também as funções lambda que são muito utilizadas por tornarem o código mais limpo diminuindo assim a complexidade de leitura do código, abaixo adicionei uma tabela feita por mim para comparar as funções *lambda com* funções normais *def:*

Tabela 2 - Comparação entre funções tradicionais e lambdas.

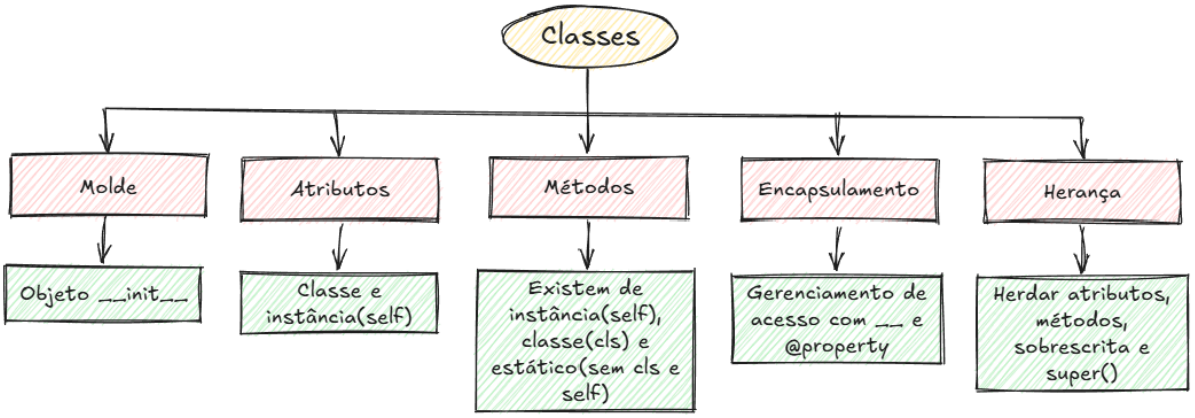
	def	lambda
Nome	obrigatório	anônima
Corpo	vários comandos	Uma única expressão
return	explícito	implícito
Uso	lógica geral	argumento curto

Fonte: Autoria própria feita no Excalidraw.

- **Programação Funcional**: No vídeo o professor deu uma breve introdução em conteúdos de programação funcional como funções dentro de funções, permitindo a captura da variável de escopo.

O segundo vídeo também deu uma explicação sobre classes e OO(Orientação a Objetos), que é de extrema importância pois é um dos principais paradigmas de construção de código utilizado nos dias de hoje. Desenvolvi a Imagem 3 para facilitar meu entendimento e apresentar os principais conceitos:

Imagem 3 - Mapa mental sobre classes e OO.



Fonte: Autoria própria feita no Excalidraw.

Seguindo a mesma lógica de explicação do restante do relatório agora vou explicar o mapa mental tópico a tópico:

- **Classes(Moldes)**: Classes em Python são “modelos” ou “moldes” que definem dados (atributos), seja no *__init__* ou como atributos de classe (diretamente no escopo da classe), e métodos de objeto, permitindo criar várias instâncias/objetos com características próprias, mas compartilhando a mesma estrutura e funcionalidades. Por isso, elas são a base da OO, possibilitando organização, reutilização e clareza do código.
- **Atributos**: Existem dois tipos principais o de instância que permite que cada objeto instanciado tenha os seus próprios e os atributos de classe declarados no próprio escopo da classe sem estarem dentro de uma função, eles compartilham os atributos entre todos os objetos.

```python
# Exemplo retirado do video 2
class Contador:
    contador = 0            # atributo de classe

    def inc_maluco(self):
        self.contador += 1  # lê da classe, escreve na instância
        return self.contador

# SAIDA:
# Usando Contador.inc() três vezes saiu → 1, 2, 3 Usando Contador.dec() três
vezes saiu → 2, 1, 0 Usando c1.inc_maluco() duas vezes saiu → 1, 2
# inc_maluco imprime 1 porque a classe voltou a 0, isso acontece porque a
leitura volta pro topo
```

- **Métodos**: Na aula foram mostrados três tipos de métodos o de instância com self que seguem a mesma lógica dos atributos de instância onde cada um dos objetos tem seu *self* próprio recebido, também tem os métodos de classe

(*@classmethod* + *cls* como parâmetro) que podem ser utilizados sem a necessidade de criar um objeto e por fim temos os métodos estáticos(*@staticmethod*) que também podem ser chamados sem a criação de objetos eles não têm acesso aos atributos da classe por isso dependem de parâmetros externos.

```python
# Exemplo retirado do video 2 é a continuação do bloco de código passado
    @classmethod
    def inc(cls):
        cls.contador += 1
        return cls.contador

    @classmethod # indica que é um metodo de classe
    def dec(cls): # cls=classe
        cls.contador -= 1
        return cls.contador

    @staticmethod
    def mais_um(n):# aqui necessita de parametro externo
        return n + 1
```

- **Encapsulamento**: O conceito de OO que foi brevemente comentado na aula, permite o gerenciamento de acesso dos atributos da classe o *Python* não possui um tipo *private* real como o Java tem, mas ele funciona pelo menos para bloquear o acesso normal de fora da classe(da para burlar isso com *_NomeClasse__NomeAtributo* por isso falei que não é um *private* real), para os métodos usa-se o *@property* que permite o acesso como atributo, com o *@nomeDoMetodo.setter* é possível fazer troca do valor do atributo *private* com o método *@property* de fora da classe.

```python
# Exemplo retirado do video 2
class Produto:
    def __init__(self, nome, preco=1.99, desc=0):
        self.nome = nome
        self.__preco = preco
        self.desc = desc

    @property
    def preco(self):
        return f'R$ {self.__preco}'

    @preco.setter # aqui o setter public usa o preco private pra trocar de fora
    def preco(self, novo_preco):
        if novo_preco >= 0:
```

```
            self.__preco = novo_preco

p2 = Produto('Caderno', 14, 0.5)
p2.preco = -17.99 # setter rejeita porque é negativo

print(p2.nome, p2.preco, p2.desc)

# SAIDA: Caderno R$ 14 0.5
```

- **Herança**: Ela permite a reutilização de código permitindo que os objetos de uma classe utilizem os métodos, funções e atributos de uma classe pai facilitando muito na construção do código, agora sobre algumas propriedades como a de sobrecarga que o Python não possui nativamente por conta da tipagem dinâmica, para substituí-la usam-se **args*(para argumentos sem nome) e ***kwargs*(para argumentos nomeados), outra propriedade a de sobrescrita, na qual o *super().nomeDoMetodo()*permite chamar a versão do pai de dentro da sobrescrita.

Python
```
# Exemplo retirado do Video 2
class Carro:
    def __init__(self):
        self.__velocidade = 0

    @property
    def velocidade(self):
        return self.__velocidade

    def acelerar(self):
        self.__velocidade += 5 # o padrao da classe é aumentar 5 na velocidade
        return self.__velocidade

class Uno(Carro): # não muda nada da pai
    pass

class Ferrari(Carro):
    def acelerar(self):
        super().acelerar()
        return super().acelerar() # aqui ele usa o super pra chamar duas vezes
a def duplicando assim  a velocidade

# Deixei sem os prints para não ocupar tanto espaço

# SAIDA: no Uno: 5 10 15 na Ferrari: 10 20 30
```

3. Conclusão

Os vídeos me ensinaram muitas possibilidades de utilização do *Python* como manipular código para criar, organizar, decidir, repetir, reutilizar e herdar, tudo isso dentro de um projeto. O mais importante é que entendi como utilizar esta linguagem que é umas das mais utilizadas para o desenvolvimento de *ML*(*Machine Learning*) e IA(Inteligência Artificial) no mundo, com isso tenho a base para poder aprender as bibliotecas *TensorFlow*, *PyTorch*, *NumPy* e *Pandas* e assim começar a ensinar o computador a encontrar padrões para resolver problemas.

4. Referências

Vídeo 1 - PYTHON 3 Curso Rápido 🐍 Parte #1 2020 - 100% Prático!
PYTHON 3 Curso Rápido 🐍 Parte #1 2020 - 100% Prático!

Vídeo 2 - PYTHON 3 Curso Rápido 🐍 Parte #2 2020 - 100% Prático!
PYTHON 3 Curso Rápido 🐍 Parte #2 2020 - 100% Prático!